

Class design — what classes will you write, and what single job does each own? No one class doing everything.

Main: main loop, it calls methods from the TUI class

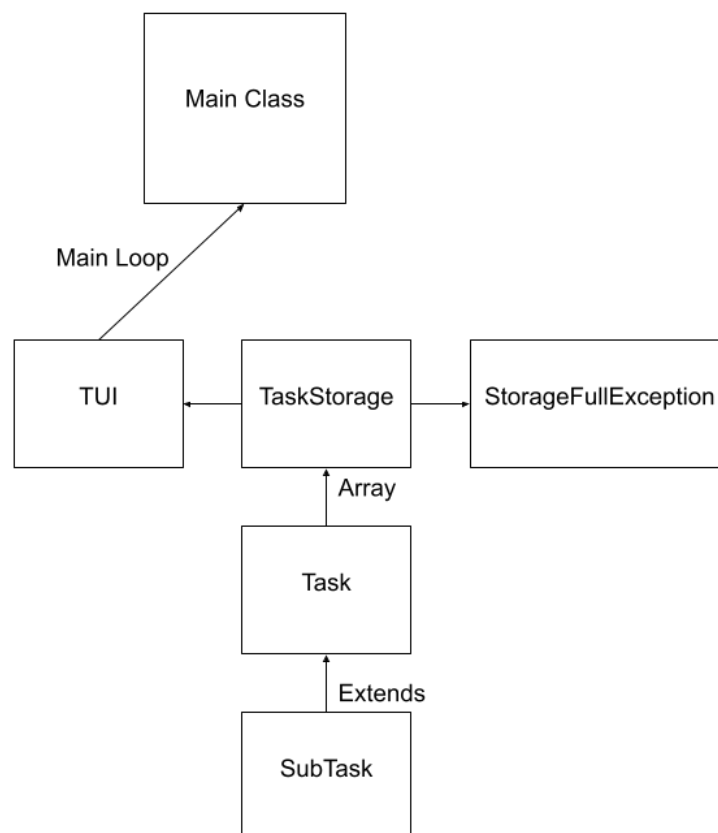
TUI: has all the methods for displaying and receiving input through the terminal

TaskStorage: Stores all the tasks in an array

Task: Primary task class, stores data for a task

SubTask: Secondary task class, extends task and stores the task it belongs to as well

System diagram — *required*. A single picture showing the main pieces of your system and how they connect: which parts talk to which, and what moves between them. It does not have to be UML or any formal notation — boxes and arrows are fine, and a photo of a legible whiteboard or hand sketch counts. The test is whether a reader can see the shape of your system at a glance without reading the rest of the spec. Label each box with the class it stands for.



Data & state — for each class, what data does it hold (fields and their types), and which collection or structure stores the core data — and why that one?

All data is private.

Main : Creates a TaskStorage instance. It also passes the TaskStorage instance into the TUI constructor.

- tui: TUI instance.

TUI :

- storage: TaskStorage, a reference passed in through the constructor
- scanner: Scanner, for reading user input
- It doesn't own any task data. It only asks TaskStorage for tasks and prints them, which keeps display logic separate from data logic.

TaskStorage:

- Task[] taskArray of size 25
- size: integer Number of currently stored tasks.
- new_task_id: integer, stores the next tasks ID to ensure no duplicates.

Task:

- id: integer the task id. Must be unique.
- description: String that holds the task description
- due_date: Date using java.util.Date

SubTask:

- Same data as Tasks
- parent_task_id: integer, Must match an existing id. Cannot belong to a subtask.

StorageFullException: extends Exception.

Method signatures — for each class, the public methods: name, parameters and types, return type, and a one-line behavior. This is the contract a teammate or GenAI builds against.

Main:

public static void main(String[] args); Main loop.

TUI:

- public TUI(TaskStorage storage, Scanner scan); Constructor.
- public void printTaskList(); Prints list of all tasks in storage.
- public void handleUserAction(); Asks the user for action (add, edit, remove) and calls the appropriate methods based on that.
- public void handleAddTask(); Asks the user for new task description and due date; Then calls the storage's add method
- public handleAddSubtask(); Asks the user for task description and due date and parent task id. Parent task cannot be a subtask.
- public handleRemoveTask(); Asks the user for task id to remove then calls TaskStorage remove. If it is invalid, prompt the user to enter a new id.

- public handleEditTask(); Asks the user for task id to edit. Then asks for a new description and date. If the description is empty then do not edit it. If the date is empty do not edit it.

TaskStorage:

- public TaskStorage(); Constructor creates an array of tasks size 25.
- public boolean add(Task task); Adds the task to the array. Throws StorageFullException if array is full. If subtask parent does not exist or parent is also a subtask then returns false.
- public boolean remove(Task task); removes the task from the array. Returns false if failed. (invalid task)
- public boolean editDescription(Task task, String new_description); Returns false if description is empty or task is null.
- public boolean editDate(Task task, Date new_due_date); public Task getTaskFromID(integer id); Returns tasks contents. Returns null if the task does not exist.

Task:

- public Task(integer id, String description, Date due_date); Constructor. ID uniqueness is verified by TaskStorage.
- public String getDescription(); getter for description
- public boolean setDescription(String description); setter for description, if string is empty return false.
- public String getDate(); getter for date
- public boolean setDate(Date new_due_date); setter for date

SubTask:

- Same as task
- public integer getParentTaskId(); getter for parent task id

StorageFullException: Only the methods extended from Exception

Where validation lives — take each bad-input case from your scope: which class/method catches it, and what happens (re-prompt, exception, default)? Every named bad input should map to a defined response.

Main:

TUI: Invalid user inputs detected by the Scanner, re-prompts the user for that input or leave blank to cancel.

TaskStorage: User tries to add another task but has already reached maximum size of the array, return Throws exception. Tries to remove a task when the array is null.

Task: Invalid characters inside date input, prompts error message and prompts user to enter again. The only accepted format is (MM/DD/YYYY) anything else is considered invalid. MM can be from 01-12, DD can be from 01-31 for long months, 01-30 for regular months, February can be 01-29 if leap year, else 01-28.

SubTask: Invalid parent task ID, subtask id's and non-existent id's are considered invalid, prompts error messages and asks the user to re-prompt.

Test plan — for each public method, one normal case and one bad-input case: input and expected result, in plain English.

Main: no tests

TUI:

- handleAddTask and handleAddSubTask
 - Normal: correct date and normal description → new task added
 - Bad: storage is already full → prompts the user to remove a task first
- handleRemoveTask Task
 - Normal: valid task ID input → task gets removed successfully
 - Bad: invalid task ID input → prompts user for new ID or exit remove process

TaskStorage:

- handleEditDescription & handleEditDate
 - Normal: user enters a valid Task ID → prompts user for new date or description
 - Bad: user enters a invalid Task ID → Prompts user for new ID or exit edit attempt

Task: takes no input from user

SubTask: takes no input from user

Division of work — who in your project owns which classes or pieces, and how will you integrate them into one working program?

Main: Abdullah

TUI: Abdullah

TaskStorage: Oscar

Task: Vihaan

SubTask: Vihaan

For tests. Whoever does the main class also does tests for it.

Revised scope — what changed since your scope doc, and why — and where did each change originate (group discussion, GenAI, your own reflection)?

The scope doc has changed a lot as our design becomes more concrete and we get into the specifics of the project. We added new task IDs, capped the storage at 25 and added a `StorageFullException` to handle overflow, and split the program into multiple different classes, `Main`, `TUI`, and `TaskStorage`, so that the input, stored data, and entry point are separated. We also added editing for the description and date.

`Main`: unchanged

`TUI`: we added handler methods instead of directly calling the remove and add methods.

`TaskStorage`: The class wasn't explicitly stated before the spec process.

`Task`: mainly unchanged

`SubTask`: mainly unchanged